STUDIJNÍ SKRIPTA / OSTRAVSKÁ UNIVERZITA

Architektury počítačů umělé inteligence

Jednotný, tematicky uspořádaný výklad k devíti přednáškovým prezentacím.

TÉMATA

CPU, GPU, TPU, VPU, NPU · ASIC a FPGA · SoC a SoM · Jetson, Coral · Edge AI a IoT · HPC · kvantové počítače

FORMA

Definice princip porovnání praktické příklady kontrolní otázky

PODKLAD

Prezentace 7APAI_01, 7APAI_02_s, 7APAI_03, 7APAI_ASIC, 7APAI_FPGA, 7APAI_Coral, 7APAI_Jetson_Nano, 7APAI_IoT a 7APAI_Mob

Jak rozlišovat původ: běžný výklad syntetizuje přednášky. Modré boxy s názvem **Vysvětlení navíc** představují výkladové doplnění. Žluté boxy označují **upřesnění či upozornění**. Číselné parametry konkrétních produktů jsou historickými příklady z prezentací, nikoli aktuálním nákupním srovnáním.

Zpracováno: říjen 2026 · Studijní materiál, nikoli oficiální publikace vyučujícího.

Obsah

Pro studium je vhodné postupovat od principů k architekturám a teprve pak ke konkrétním zařízením.

Proč AI potřebuje specifický hardware 3

Jak správně měřit výkon a efektivitu 4

CPU a jejich instrukční architektury 5

GPU, TPU, NPU a VPU – přehled akcelerátorů 6

Integrované obvody a ASIC 8

FPGA – programovatelný hardware 9

SoC, SoM, MCU a MPU 10

Edge AI a nasazení modelů 11

NVIDIA Jetson Nano 12

Google Coral a FreeRTOS 13

Internet věcí, komunikace a průmysl 4.0 14

AI v mobilních zařízeních 16

Superpočítače a HPC pro AI 17

Kvantové počítače – potenciál a limity 18

Srovnání a výběr architektury 19

Kontrolní otázky a odpovědi 20

Slovníček, poznámky a zdroje 23

DOPORUČENÝ ZPŮSOB UČENÍ

1. Nejprve si vysvětli pojmy vlastními slovy. 2. U každé architektury určete, co je programovatelné, co je pevně dáno a jak se měří výkon. 3. Nakresli vlastní diagram. 4. Zkus kontrolní otázky bez nahlížení do odpovědí.

KAPITOLA 01

Proč AI potřebuje specifický hardware

Od matematických operací k požadavkům na výpočetní systém.

Ze zdrojů: 7APAI_01, snímky 2–11, 22–27.

1.1 AI, strojové učení a hluboké učení

Umělá inteligence (AI) je širší oblast metod umožňujících vykonávat úlohy vyžadující inteligentní zpracování informací. **Strojové učení (ML)** zahrnuje metody, které odvozují pravidla nebo parametry z dat. **Hluboké učení (DL)** je podmnožinou ML založenou zejména na vícevrstvých neuronových sítích.

Prezentace zdůrazňují, že AI nemusí obsahovat jednu extrémně komplikovanou operaci: typicky opakuje obrovské množství relativně jednoduchých výpočtů. V neuronové síti jsou to zejména násobení a sčítání při práci s vektory, maticemi a tenzory.

Hybridní ML podle výkladu přednášek kombinuje data, znalosti a kontext; například statistický prediktor doplněný pravidly nebo znalostní reprezentací. Jde o přístup k návrhu řešení, nikoli o konkrétní typ procesoru.

1.2 Trénování vs. inference

Trénování (training)	Inference (nasazení modelu)
Parametry modelu se mění podle trénovacích dat.	Parametry jsou zpravidla pevné; pro nový vstup se počítá výsledek.
Typicky vysoké nároky na výpočetní výkon, paměť, datové pipeline a komunikaci.	Často je kritická latence, energie, cena za dotaz a velikost modelu.
Obvykle GPU/TPU nebo distribuovaný cluster.	CPU, GPU, NPU, VPU, Edge TPU či mikrokontrolér podle úlohy.

Příklad: síť pro rozpoznání člověka v obraze se vytrénuje na velkém souboru dat, ale na dveřní kameře může běžet pouze její inference. Není nutné instalovat trénovací cluster do každé kamery.



Zjednodušený datový tok inference. Při trénování navíc existuje výpočet chyby a aktualizace parametrů.

1.3 Výpočetní infrastruktura AI

- **Výpočet:** CPU a akcelerátory provádějí operace.
- **Paměť a úložiště:** drží data, váhy, mezivýsledky a checkpointy.
- **Síť:** přenáší data a synchronizuje více uzlů; u lokálního edge zařízení může mít menší roli.

VYSVĚTLENÍ NAVÍC

Typické jádro výpočtu je násobení s akumulací **MAC**: akumulátor ← akumulátor + (a × b). Například výpočet neuronu $z = w_1x_1 + w_2x_2 + \ldots + b$ obsahuje mnoho MAC. Akcelerátory nejsou „inteligentní“ samy o sobě; jejich zapojení odpovídá operacím, které algoritmy často vyžadují.

ZAPAMATUJ SI

Zásadní rozdíl: **architektura** říká, jak hardware pracuje; **workload** říká, co na něm počítáme. Dobrý výběr vyžaduje obojí.

KAPITOLA 02

Jak správně měřit výkon a efektivitu

Co vlastně znamenají FLOPS, TOPS, latence, propustnost a paměťová propustnost.

Ze zdrojů: 7APAI_01, sn. 3–4, 14–17, 30–49; 7APAI_03, sn. 4, 9, 16–18.

2.1 Latence, throughput a paralelismus

- **Latence:** čas pro jeden požadavek; typicky ms/inference.
- **Propustnost (throughput):** počet úloh zpracovaných za sekundu, např. obrázků/s.
- **Paralelismus:** současné provádění nezávislých operací nebo částí výpočtu.
- **Škálování:** změna výkonu při přidání jader, akcelerátorů nebo serverů.

Nízká latence a vysoká propustnost nejsou totéž. Velká GPU může dávkovým zpracováním (batching) výrazně zvýšit propustnost, aniž by zkrátila odezvu jednotlivého požadavku.

2.2 FLOPS, TOPS a přesnost

Metrika	Co měří	Pozor
FLOPS / GFLOPS / TFLOPS	Operace s plovoucí řádovou čárkou za sekundu.	Záleží na FP64, FP32, FP16, BF16 aj.
TOPS	Biliony (10^{12}) operací/s; často celočíselné INT8.	Bez uvedení datového typu a metodiky nejde výkon srovnávat.
Rpeak	Teoretický špičkový výkon.	Není to skutečný výkon aplikace.
Rmax	Naměřený výkon benchmarku LINPACK.	Měří konkrétní výpočet; nikoli automaticky výkon DL.

Přednášky uvádějí například **4 TOPS (INT8)** u Google Edge TPU a přibližně **0,5 TFLOPS (FP16)** u Jetson Nano. Tyto hodnoty se **nedají přímo porovnat**: liší se typy operací, přesnost, architektura i způsob vykazování.

VYSVĚTLENÍ NAVÍC – PAMĚŤOVÁ ZEĎ

Pro výpočet záleží nejen na maximálních operacích/s, ale i na rychlosti přísunu dat. **Operační intenzita** = počet operací / počet přečtených a zapsaných bajtů. Přibližný horní limit: **výkon ≤ min(špičkový výkon čipu, propustnost paměti × operační intenzita)**. Tento princip se nazývá roofline model.

2.3 Energie a celkové náklady

Mnoho edge zařízení se posuzuje pomocí **wattů**, **energie na inferenci** a nároků na chlazení. Hrubý odhad: E = P × t. Zařízení s příkonem 2 W, které úlohu provede za 0,5 s, spotřebuje na tuto operaci při tomto příkonu asi 1 J. Celková spotřeba se však liší podle dalších komponent a provozního režimu.

U velkých clusterů se hodnotí výkon na watt (např. žebříček Green500), infrastruktura chlazení i cena přenosu dat. Vysoké TOPS bez dostatečné paměti nebo kompatibility modelu neznamenají kvalitní řešení.

> **VYSVĚTLENÍ NAVÍC – OMEZENÍ PARALELIZACE**
>
> Amdahlův zákon pro urychlení pomocí n jednotek: **S(n) = 1 / ((1 p) + p/n)**, kde p je paralelizovatelný podíl práce. Pokud lze paralelizovat 90 % programu, ani nekonečný počet jader nepřinese více než 10× zrychlení.

2.4 Otázky, které položit nad každým benchmarkem

- Jaký datový typ a přesnost jsou použity?
- Jde o teoretické maximum, měření kernelu nebo aplikace?
- Jaká je velikost dávky (batch) a modelu?
- Započítává výsledek přenos do zařízení a z něj?
- Je dosažený výkon limitován výpočty, pamětí, nebo sítí?

KAPITOLA 03

CPU a jejich instrukční architektury

Proč jsou univerzální procesory stále nezbytné i v AI systémech.

Ze zdrojů: 7APAI_02_s, sn. 6–8, 23–31; 7APAI_03, sn. 10–18.

3.1 CPU: řídicí a univerzální část systému

CPU je univerzální procesor s rozsáhlou sadou instrukcí, hierarchií cache a řízením toku programu. V AI zajišťuje přípravu a přesuny dat, operační systém, I/O, serializační části programu i vlastní inferenci menších modelů. Oproti specializovaným akcelerátorům nabízí širokou kompatibilitu a vysokou flexibilitu.

3.2 ISA vs mikroarchitektura

ISA (Instruction Set Architecture) je instrukční rozhraní viditelné programátorovi a překladači. **Mikroarchitektura** je konkrétní implementace (pipeline, cache, pořadí vykonávání, predikce větvení). ARM, MIPS a x86 jsou rodiny ISA; dvě CPU se stejnou ISA se výkonem i spotřebou mohou zásadně lišit.

Rodina	Typické charakteristiky a příklady v podkladech
ARM	RISC tradice, významné využití v mobilech, MCU, SoC, serverech; Cortex-M/A, Apple Silicon, Ampere.
MIPS	RISC ISA; historicky embedded systémy, konzole a síťová zařízení. Přednášky ukazují MIPS M62xx a pipelining.
x86	Významné PC a servery; Intel Meteor Lake, AMD EPYC a Ryzen. Moderní implementace jsou interně velmi pokročilé.

3.3 RISC, CISC, pipeline a heterogenní jádra

RISC a **CISC** jsou historické koncepce ISA. Není správné odvozovat výkon nebo energetickou účinnost pouze z označení ARM/RISC proti x86/CISC. Rozhoduje konkrétní čip, výrobní technologie a software.

Pipelining rozděluje zpracování instrukce do etap (např. načtení, dekódování, vykonání, zápis). Více instrukcí se současně nachází v různých etapách; zvyšuje se průchodnost, nikoli nutně latence jedné instrukce.

Heterogenní CPU kombinují výkonná a úsporná jádra. Přednáška zmiňuje Intel Meteor Lake (dlaždicové uspořádání) a Apple Silicon s kombinací CPU, GPU a Neural Engine.

> **PŘÍKLAD Z PREZENTACÍ**
>
> Apple M2 Ultra demonstruje integraci výkonných a úsporných CPU jader, GPU, paměti a Neural Engine; AMD EPYC a Intel Xeon demonstrují serverové platformy. Konkrétní počty jader, frekvence a procesy na slidech se vztahují k uvedeným generacím, nikoli k „typickému ARM či x86“.

3.4 Paměťová hierarchie

Registry jsou nejrychlejší, ale velmi malé; následují cache L1/L2/L3, operační paměť a úložiště. Pokud data opakovaně čteme z pomalé paměti, může se snížit efektivní výkon i u výkonného CPU. Pro výpočty AI je proto důležitá lokálnost dat, vektorové instrukce a dostupnost efektivních knihoven.

KAPITOLA 04

GPU, TPU, NPU a VPU – přehled akcelerátorů

Rozdíl mezi masivním paralelismem a výpočty prováděnými účelovými jednotkami.

Ze zdrojů: 7APAI_01, sn. 27–49; 7APAI_Mob, sn. 4, 13–22.

4.1 GPU (Graphics Processing Unit)

GPU vznikly pro grafické výpočty a disponují mnoha paralelně pracujícími výpočetními jednotkami. V DL se používají k operacím s tenzory, zejména násobení matic, konvolucím a gradientům. Jsou důležité pro trénování velkých modelů i pro inferenci. Software může používat CUDA, knihovny neuronových sítí a frameworky typu PyTorch nebo TensorFlow.

Efektivita GPU roste, když je práce dost velká a dobře paralelizovatelná. Krátké sekvenční úlohy mohou trpět režiemi spouštění kernelů a přesuny dat. V prezentacích: NVIDIA A100/H100 a embedded řada Jetson.

4.2 TPU (Tensor Processing Unit)

TPU označuje specializované akcelerátory zaměřené na tensorové výpočty, často založené na masivní práci se součiny a akumulacemi. Je nutné odlišit **cloudové TPU** pro výkonné trénování/inferenci a **Google Edge TPU** pro úspornou lokální inferenci v podporovaných modelech.

4.3 NPU a VPU

NPU (Neural Processing Unit) je obecnější název specializované jednotky pro neuronové sítě, typicky v mobilních SoC a embedded systémech. **VPU (Vision Processing Unit)** je zaměřena na obraz, kamery a počítačové vidění; v podkladech jsou Intel Movidius Myriad 2 a Myriad X.

Movidius Myriad 2 podle prezentace obsahuje vektorové jednotky SHAVE a cílí na velmi nízký příkon; Myriad X přidává Neural Compute Engine a větší integrační možnosti senzorů. Technologie Intel OpenVINO je uvedena jako součást vývojového ekosystému.

Jednotka	Silná stránka	Typický kompromis
CPU	Univerzálnost, řízení, snadná kompatibilita.	Nižší efektivita u masivně paralelních matic.
GPU	Programovatelný paralelní výkon; trénování i inference.	Spotřeba; režie; přenos dat.
TPU / Edge TPU	Vysoce efektivní podporované tensorové operace.	Omezení modelů, datových typů, nástrojů.
NPU	Úsporná AI uvnitř SoC.	Silná závislost na ovladačích a podporovaných operátorech.
VPU	Inferování a zpracování obrazu na edge.	Specifické doménové omezení a ekosystém.

VYSVĚTLENÍ NAVÍC – DĚLENÍ PRÁCE

CPU koordinuje aplikaci, GPU může zpracovat velké tensorové výpočty, specializovaná NPU zrychlí úspornou inferenci a ISP předzpracuje obraz z kamery. V jedné aplikaci mohou pracovat současně. Akcelerátor není náhrada celého počítače.

POZOR NA ZOBECNĚNÍ

Tvrzení „TPU má vždy menší spotřebu než GPU“ není univerzální. Vhodnější je porovnat **energii na skutečně vykonanou inferenci** u daného modelu, přesnosti a dávky.

KAPITOLA 05

Integrované obvody a ASIC

Od křemíkového waferu k obvodu navrženému pro konkrétní účel.

Ze zdrojů: 7APAI_ASIC, sn. 2–20; 7APAI_01, sn. 50–52.

5.1 Co je integrovaný obvod

Integrovaný obvod obsahuje elektronické prvky vytvořené na polovodičovém čipu. Prezentace rozlišuje analogové a digitální, monolitické a hybridní obvody a uvádí stupně integrace SSI, MSI, LSI, VLSI, ULSI. Tyto zkratky popisují historické kategorie hustoty integrace; dnešní čipy je často mnohonásobně překračují.

5.2 ASIC: Application-Specific Integrated Circuit

ASIC je integrovaný obvod navržený pro konkrétní účel nebo aplikaci. Výhoda je možnost přizpůsobit datové cesty, paměť a řízení dané práci; nevýhoda nákladného návrhu a nemožnosti zásadně změnit hotovou hardwarovou logiku pouhým přepsáním konfigurace.

5.3 Jak se navrhují ASIC

Přístup	Princip
Full custom	Detailní návrh na úrovni tranzistorů a všech relevantních masek; nejvyšší nároky na vývoj.
Standard cells	Použití knihoven hotových logických buněk (hradla, registry); syntéza a fyzický návrh.
Gate array / MPGA	Předpřipravené pole buněk, jejichž propojení se dokončuje maskami.
Structured/platform ASIC	Připravené bloky a IP jádra, přizpůsobení konkrétnímu systému.

IP blok představuje znovupoužitelnou část návrhu, například RAM, DSP, UART, procesorové jádro nebo rozhraní. Masky jsou fyzické výrobní vrstvy používané při fotolitografii. Výrobní náklady čipu, náklady na návrh (NRE) a objem výroby se musí posuzovat společně.

5.4 ASIC vs FPGA při rozhodování

Při malých sériích a rychlých změnách často vychází lépe FPGA; při velmi velkých sériích a stabilní specifikaci může být výhodnější ASIC. Prezentace FPGA uvádí hranici 400 000 kusů jako příklad; ve skutečnosti **neexistuje jedna univerzální hranice**: závisí na technologickém procesu, velikosti čipu, ceně vývoje i cílové spotřebě.

> **PRAKTICKÝ PŘÍKLAD**
>
> Obvod pro jedinou rutinu v milionu levných senzorů může být vhodný pro ASIC. Pro vývoj prototypu průmyslového akcelerátoru s častými změnami bývá výhodnější FPGA. TPU je příkladem účelové akcelerace; není však pravda, že každý ASIC musí být TPU.

KAPITOLA 06

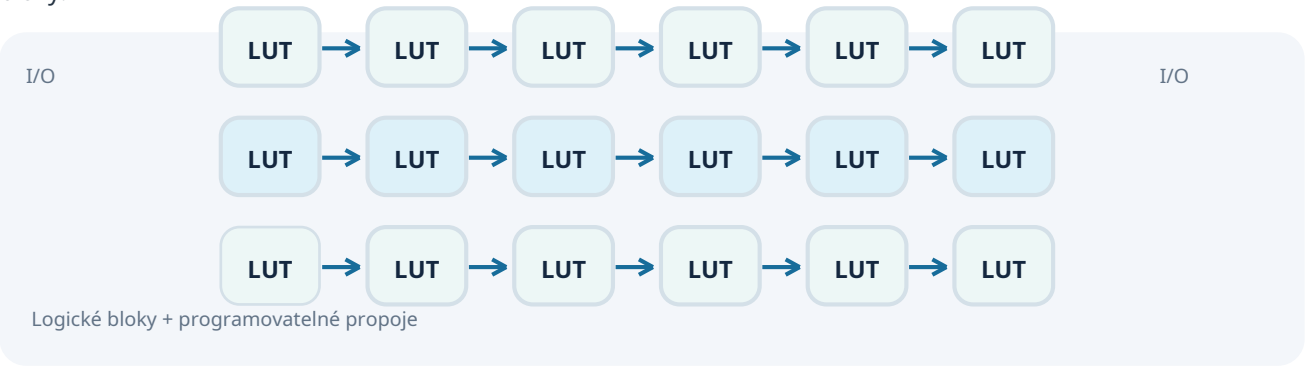
FPGA – programovatelný hardware

Struktura logických bloků, technologie konfigurace a vývojový postup.

Ze zdrojů: 7APAI_FPGA, sn. 2–29; 7APAI_ASIC, sn. 18–19.

6.1 Co znamená „programovat FPGA“

FPGA (Field-Programmable Gate Array) umožňuje **nakonfigurovat strukturu digitálního obvodu** po jeho výrobě. Nejde pouze o spuštění běžného programu na CPU. Konfigurační data stanovují funkce logických bloků a jejich fyzické propojení. FPGA může realizovat paralelní pipeline, vlastní datové cesty nebo celé systémové bloky.



Schematický pohled: bloky LUT a konfigurovatelná propojení. Skutečné FPGA navíc obsahuje registry, RAM, DSP bloky, hodiny a rozhraní.

6.2 Stavební prvky

- **LUT (Lookup Table):** malá pravdivostní tabulka implementující logickou funkci. Například čtyřvstupová LUT může kódovat 16 možných kombinací vstupů.
- **Flip-flop / registr:** ukládá bit mezi hodinovými takty; umožňuje sekvenční logiku.
- **Routing / přepínače:** přenos dat mezi bloky; přednáška rozlišuje PIP a SMIP.
- **DSP bloky a paměť:** realizace násobení a akumulací, buffery a datové fronty.

6.3 Volatilní a nevolatilní konfigurace

Varianta	Vlastnosti	Kompromisy
SRAM FPGA	Konfigurace v energeticky závislé SRAM; lze opakovaně rekonfigurovat.	Typicky vyžaduje načtení konfigurace po startu; ochrana bitstreamu.
Flash / EEPROM FPGA	Nevolatilní ukládání konfigurace, možné přeprogramování.	Odlišné výrobní a výkonové vlastnosti, omezení počtu zápisů.
Antifuse FPGA	Spoj se jednorázově fyzicky vytvoří (OTP).	Trvalá konfigurace; nelze rekonfigurovat.

6.4 HDL a vývojový tok

Základními jazyky pro popis hardwaru jsou **VHDL** a **Verilog**; SystemVerilog rozšiřuje jazyk i verifikaci. Typický tok: specifikace RTL návrh simulace syntéza place & route časová analýza bitstream programování a testování. HDL popisuje logickou strukturu a časování; není vhodné ho chápat jako obyčejný sekvenční program.

> **VYSVĚTLENÍ NAVÍC – KDE FPGA ZÍSKÁVÁ VÝKON**
>
> Při streamování obrazu lze vytvořit hardware, který každý takt přijímá nový pixel. Jednotlivé fáze pipeline pracují souběžně nad různými pixely. Díky tomu může být throughput vysoký i bez vysoké frekvence jádra. Latence průchodu jedním pixelem může stále odpovídat více taktům.

6.5 Kdy použít FPGA pro AI

Hodí se pro deterministické zpracování streamů, vlastní nízkobitovou aritmetiku, předzpracování obrazu a prototypování akcelerátorů. Limity tvoří náročnost RTL/HLS návrhu, kapacita paměti a logiky, délka vývoje a podpora frameworků.

KAPITOLA 07

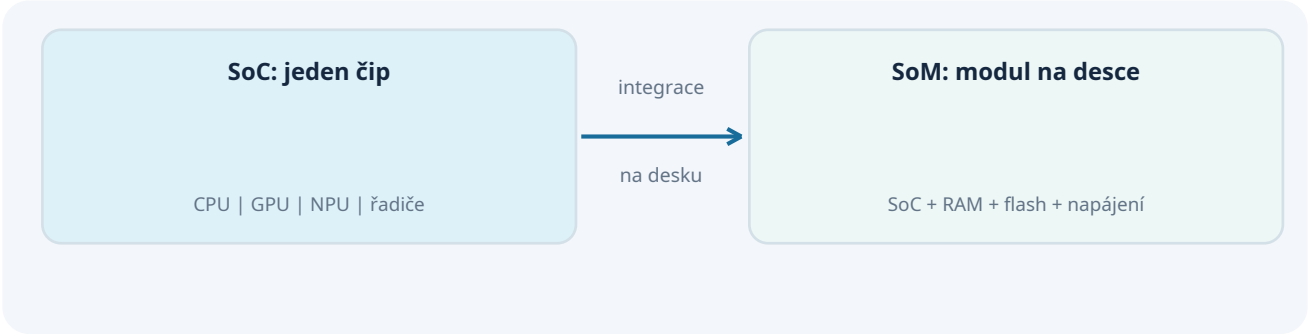
SoC, SoM, MCU a MPU

Pozor na čtyři pojmy, které popisují různé vrstvy systému.

Ze zdrojů: 7APAI_02_s, sn. 2–6, 12–23, 33–39.

7.1 System-on-Chip vs System-on-Module

SoC (System-on-Chip) integruje na jednom čipu CPU, řadiče, periférie a často GPU, NPU, DSP nebo modem. **SoM (System-on-Module)** je fyzický modul/deska s SoC, pamětí, napájecími obvody a často i konektivitou. SoM se obvykle osazuje na nosnou desku zařízení (carrier board).



Zjednodušené rozlišení: SoC je čip; SoM je obvykle modul s čipem a podpůrnými součástmi.

SoC	SoM
Vysoká integrace uvnitř pouzdra čipu.	Sjednocuje více součástek na modulu.
Nižší možnosti výměny jednotlivých bloků.	Modul lze v některých návrzích vyměnit nebo nahradit.
Příklady: Kirin 9000, Apple M2, NXP i.MX 8M.	Příklady: Coral SoM, Jetson Nano SoM, CL-SOM-iMX8Plus.

7.2 MCU vs MPU

MCU (mikrokontrolér) zpravidla spojuje procesorové jádro s vnitřní pamětí a periferiemi pro řízení zařízení v reálném čase; často používá RTOS nebo běží bez OS. **MPU (mikroprocesor)** typicky používá bohatší paměťový a operační systém (např. Linux). MCU/MPU popisují charakter výpočetního jádra/systému, nikoli alternativu k SoC či SoM.

Přednášky uvádějí **PSoC 5LP** s ARM Cortex-M3, **STM32** s Cortex-M7 a **NXP i.MX** v embedded AI. Tyto příklady se liší výkonnostní třídou i rozsahem periferií.

7.3 Příklady integrace

- **Apple M2 / M2 Ultra:** CPU, GPU, paměťové rozhraní a Neural Engine pro obecné a AI výpočty.
- **Raspberry Pi (Broadcom BCM2711 v podkladech):** SoC jako jádro malého počítače.
- **Coral SoM:** SoC NXP i.MX 8M, RAM, eMMC a Edge TPU na modulu.
- **Jetson Nano:** modul s ARM CPU a NVIDIA GPU, používaný také na vývojové desce.

> **ZAPAMATUJ SI**
>
> Slova **SoC** a **SoM** nejsou konkurenční typy procesorů. SoM může obsahovat SoC, který obsahuje CPU i NPU.

KAPITOLA 08

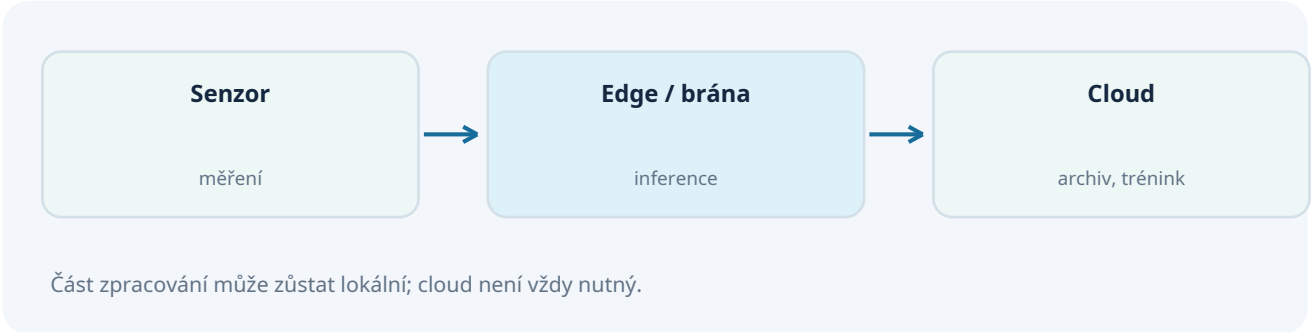
Edge AI a nasazení modelů

Proč část AI opouští datová centra a běží blízko senzorů.

Ze zdrojů: 7APAI_01, sn. 23–27, 43–49; 7APAI_IoT, sn. 3–11, 31.

8.1 Cloud, edge a zařízení

Cloud centralizuje velké výpočetní zdroje, ukládání a správu. **Edge computing** zpracovává data blíže jejich vzniku (brána, lokální server, zařízení). **On-device AI** provádí inferenci přímo na koncovém zařízení. Mezi úrovněmi je kontinuum, nikoli ostrá hranice.



Příklad rozdělení práce mezi senzor, edge bránu a datové centrum.

8.2 Výhody a nevýhody

Výhody edge	Kompromisy
Nižší latence bez cesty do cloudu.	Omezená RAM, kapacita baterie a chlazení.
Menší síťový provoz; lokální ochrana dat.	Složitější správa velkého množství zařízení.
Možnost práce při výpadku konektivity.	Aktualizace, bezpečnost modelů a nejednotný hardware.

8.3 Optimalizace modelu pro zařízení

Volba hardwaru závisí na velikosti modelu, podpoře operátorů, požadované rychlosti snímkování, RAM a energii. Nestačí, že akcelerátor teoreticky zvládá desítky TOPS: konkrétní model se musí podařit přeložit a spustit v podporovaném formátu.

> **VYSVĚTLENÍ NAVÍC – KVANTIZACE**
>
> **Kvantizace** převádí váhy a/nebo aktivace například z FP32 na INT8. Cíl: menší paměť a rychlejší inference na kompatibilním hardware. Cena: možné zhoršení numerické přesnosti a nutnost ověřit kvalitu výstupů. Často se používá post-training quantization; někdy quantization-aware training.

8.4 Praktické rozhodnutí

Kamera, která musí reagovat do desítek milisekund i při výpadku internetu, je kandidát na edge inference. Služba trénující velmi rozsáhlý model nad miliony záznamů je kandidát na datové centrum. Často nejlépe funguje kombinace: **trénink v cloudu, inference na edge, centrální monitoring**.

KAPITOLA 09

NVIDIA Jetson Nano

Konkrétní příklad embedded GPU platformy pro AI.

Ze zdrojů: 7APAI_Jetson_Nano, sn. 2–11; 7APAI_01, sn. 38–42.

9.1 Co je Jetson Nano

Jetson Nano je embedded platforma, která kombinuje čtyřjádrový procesor **ARM Cortex-A57** s GPU NVIDIA Maxwell o **128 CUDA jádrech**. Vývojové prostředí vychází z Linuxu a ekosystému NVIDIA JetPack, podporuje Python a nástroje pro práci s obrazem. Je určen pro výuku, robotiku, IoT i inferenci počítačového vidění.

9.2 Parametry uváděné v přednášce

Parametr	Hodnota v podkladech
GPU	128 jader CUDA (Maxwell), cca 0,5 TFLOPS FP16
CPU	4× ARM Cortex-A57
Paměť	4 GB LPDDR4; propustnost přibližně 25,6 GB/s
Typický příkon	Pro uváděnou variantu přibližně 5–10 W
Rozhraní	USB, HDMI, Ethernet, MIPI CSI pro kamery, GPIO
Příklady použití	Klasifikace a detekce objektů, segmentace, rozpoznávání gest, domácí automatizace

Některé údaje ve snímcích odpovídají modulu, jiné vývojové sadě. Například dostupnost **eMMC vs microSD** závisí na konkrétní variantě, a proto je třeba pro skutečný návrh ověřit přesné SKU a datasheet.

9.3 Typický výpočetní tok

Kamera → načtení snímku → předzpracování (CPU/GPU) → inference CNN na GPU → výsledek (např. detekovaný objekt) → akce. Reálnou propustnost ovlivňuje rozlišení, velikost sítě, dekódování videa a kopírování paměti.

> **SROVNÁNÍ S CORAL**
>
> Jetson nabízí programovatelnější GPU a širší spektrum modelů v rámci kompatibilního software; Coral Edge TPU staví na podporovaných úsporných modelech, často kvantizovaných na INT8. Nerozhoduje samotná hodnota FLOPS/TOPS.

KAPITOLA 10

Google Coral a FreeRTOS

Vysvětlení Edge TPU, rozdílu mezi vývojovými deskami a mikrokontrolérem.

Ze zdrojů: 7APAI_Coral, sn. 2–15; 7APAI_01, sn. 45–49; 7APAI_02_s, sn. 12, 37–38.

10.1 Co je Coral a Edge TPU

Rodina Google Coral zahrnuje vývojové desky, moduly a USB akcelerátory. Společným motivem je **Edge TPU**: specializovaný ASIC pro úspornou inferenci podporovaných neuronových modelů. Prezentace uvádí orientační výkon **4 TOPS při INT8** a energetickou efektivitu až **2 TOPS/W** pro uvedenou generaci.

10.2 Dvě odlišná zařízení

Coral Dev Board	Coral Dev Board Micro
Systém s NXP i.MX 8M, Cortex-A53, GPU a Edge TPU.	Mikrokontrolérová platforma NXP i.MX RT1176 + Edge TPU.
Má Linuxové/SoM uspořádání vhodné pro obecnější aplikace.	Má Cortex-M7 @ 800 MHz, Cortex-M4 @ 400 MHz; používá FreeRTOS.
RAM v přednášce 1/4 GB, eMMC a rozhraní pro periferie.	Přibližně 64 MB SDRAM; kamera 324 × 324 px, PDM mikrofon.
Typicky komplexnější edge počítač.	Typicky nízkopříkonové vestavěné zařízení, snímání a inference.

10.3 FreeRTOS a TinyML

FreeRTOS je operační systém reálného času pro embedded zařízení. Umožňuje plánovat úlohy s omezenými zdroji a předvídatelnými reakcemi. **TensorFlow Lite** a **TensorFlow Lite for Microcontrollers (TFLM)**, zmiňované na slidech, slouží pro nasazování kompatibilních ML modelů; konkrétní podporované operace závisejí na zařízení a runtime.

Micro má podle prezentace nízkou spotřebu (orientačně 2 W) a rozhraní pro senzory; je to vhodná demonstrační platforma pro lokální detekci vizuálních a zvukových událostí.

> **VYSVĚTLENÍ NAVÍC – CO PŘED SPUŠTĚNÍM ZKONTROLOVAT**
>
> Model musí pasovat do paměti a splňovat požadavky runtime/kompilátoru Edge TPU. Jestli část sítě běží na CPU a část na akcelerátoru, výkon i energie se mohou výrazně změnit.

KAPITOLA 11

Internet věcí, komunikace a průmysl 4.0

Jak se propojují senzory, edge zařízení, sítě a rozhodovací logika.

Ze zdrojů: 7APAI_IoT, sn. 3–31.

11.1 IoT architektura

Internet of Things (IoT) označuje síť fyzických zařízení schopných sbírat data, komunikovat a případně aktivně řídit okolí. Prezentace rozlišuje senzory (odesílají data), akční členy (reagují) a zařízení kombinující obojí. Typický řetězec: senzor lokální komunikace gateway / edge backend/cloud uživatel či akční člen.

Požadavky zahrnují sběr, ukládání, analýzu a sdílení dat, dostupnost a především zabezpečení. **CIoT** se týká spotřebitelských zařízení; **IIoT** průmyslového monitorování, energetiky, logistiky a automatizace.

11.2 Komunikační technologie v souvislostech

Technologie	Obvyklá role	Důležité hledisko
Wi-Fi (802.11)	Lokální síť s vyšší datovou rychlostí; kamery.	Vyšší spotřeba, dostupnost přístupového bodu.
Bluetooth / BLE	Krátká vzdálenost, wearables, blízké senzory.	Energeticky úsporné režimy; nelze všechny verze zaměňovat.
Zigbee	Nízkopříkonové senzory, automatizace a mesh.	Potřeba odpovídajícího koordinátoru/ekosystému.
LoRa / LoRaWAN	Dlouhý dosah, malé zprávy, nízká spotřeba.	Nízká přenosová rychlost; LoRa je rádiová vrstva, LoRaWAN síťový protokol.
Sigfox	Jednoduchá telemetrie s nízkým datovým tokem.	Závislost na pokrytí a poskytovateli.
Mobilní sítě, NB-IoT	Spravovaná konektivita na delší vzdálenost.	Tarify, pokrytí, režimy spotřeby.
Ethernet / RS-485 / SPI / I^2C	Drátová síť nebo lokální propojení komponent.	Ethernet není totéž co I^2C/SPI uvnitř zařízení.

DŮLEŽITÉ UPŘESNĚNÍ

Dosahy a rychlosti ve snímcích jsou **orientační**. Závisejí na regionu, pásmu, anténách, překážkách, výkonových limitech a konfiguraci. Například maxima 5G neznamenají běžnou odezvu pod 1 ms v libovolné síti.

11.3 CPS, multiagentní a holonické systémy

Kyberfyzikální systém (CPS) propojuje software, senzory, řízení a fyzické procesy. **Multiagentní systém** obsahuje autonomně se rozhodující jednotky; agent může být i čistě softwarový. **Holonický systém** v prezentaci popisuje výrobu tvořenou spolupracujícími jednotkami (od čidla přes stroj až po výrobní celek). V Průmyslu 4.0 tak vznikají distribuované řídicí systémy.

11.4 AI uvnitř IoT – kdy přináší přidanou hodnotu

Jednoduchý termostat nepotřebuje nutně neuronovou síť. AI dává smysl tam, kde je úloha složitější než pravidlo: diagnostika anomálií ve vibracích stroje, vizuální kontrola kvality nebo predikce poruch. Pro řízení v reálném čase musí výpočet splnit časový limit; jinak může AI systém zhoršit bezpečnost.

KAPITOLA 12

AI v mobilních zařízeních

Mobilní SoC jako souhra CPU, GPU, ISP, NPU, paměti a modemu.

Ze zdrojů: 7APAI_Mob, sn. 2–29; 7APAI_02_s, sn. 14–18.

12.1 K čemu telefon používá AI

- **Foto/video:** rozpoznání scén, segmentace, odstranění šumu, HDR, přibližování, OCR.
- **Řeč a jazyk:** asistenti, přepis a překlady.
- **Bezpečnost:** biometrie a detekce podezřelého chování.
- **Provoz telefonu:** správa spotřeby, nabíjení, teploty a priorit procesů.
- **Wearables:** detekce aktivit a pádů, monitorování signálů.

12.2 Heterogenní SoC telefonu

CPU spouští obecný kód a koordinuje procesy; GPU zpracovává grafiku a paralelní výpočty; **NPU** přebírá podporované modely; **ISP** realizuje obrazové zpracování od senzoru; DSP pomáhá se signály a modem se stará o komunikaci. Paměťová hierarchie je pro výkon stejně důležitá jako počty jader.

12.3 Příklady z prezentace

SoC / rodina	Co ukazuje
Huawei Kirin 9000	ARM CPU, GPU Mali, AI akcelerace Da Vinci, ISP a 5G modem.
Samsung Exynos	Integrace CPU, GPU, NPU, ISP a modemu do mobilního SoC.
Apple A17 / A17 Pro	Neural Engine, GPU s grafickou akcelerací, pokročilý výrobní proces.
Snapdragon 8 Gen 2 / Dimensity 9200+	Srovnání heterogenních CPU jader, GPU, paměti a modemu.

JAK ČÍST SROVNÁVACÍ TABULKY

Samotná frekvence CPU nebo počet GPU jader neurčí nejlepší mobil pro AI. V praxi se sleduje konkrétní úloha, trvalý výkon při zahřívání, energie, dostupnost API a přesnost modelu. Parametry ve snímcích se vztahují ke generacím z doby vzniku prezentací.

KAPITOLA 13

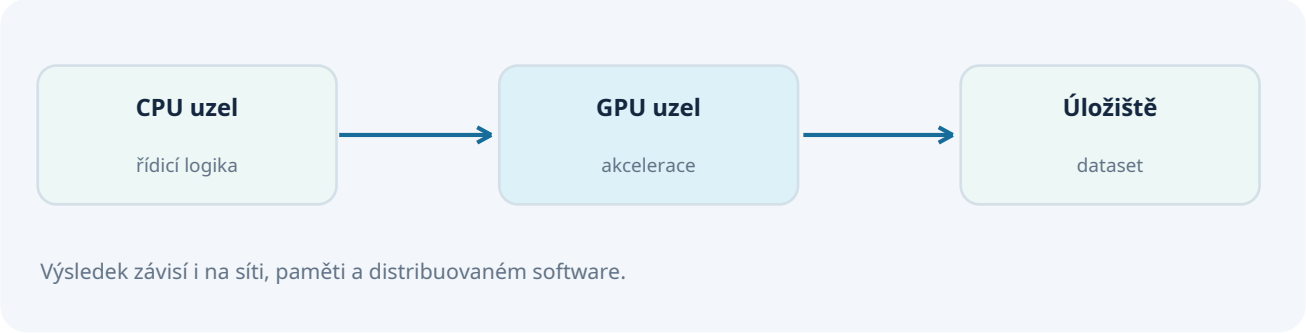
Superpočítače a HPC pro AI

Od výpočetního uzlu k masivně distribuované infrastruktuře.

Ze zdrojů: 7APAI_03, sn. 2–24; 7APAI_01, sn. 11–17.

13.1 HPC není jen silné GPU

High-Performance Computing je oblast rozsáhlých výpočetních systémů. Superpočítač obvykle tvoří mnoho uzlů s CPU, někdy GPU, vysokorychlostní sítí, paralelním souborovým systémem, plánovačem úloh a optimalizovaným softwarem. Systém řeší numerické simulace, datovou analýzu i AI.



Zjednodušený HPC celek: propojené výpočetní a datové subsystémy.

13.2 Paralelní, distribuované a vektorové výpočty

Paralelní výpočet znamená současné provádění úloh; může probíhat i uvnitř jediného čipu. **Distribuovaný výpočet** používá více propojených uzlů s oddělenou pamětí a komunikací. **Vektorový výpočet / SIMD** aplikuje stejnou operaci na více datových elementů. Tyto vlastnosti se mohou kombinovat.

Při trénování DL existuje **data parallelism** (repliky modelu na různých dávkách), **model parallelism** (části modelu na různých zařízeních) i pipeline parallelism. Síťová synchronizace a rychlost úložiště často omezují škálování.

13.3 TOP500, Green500 a LINPACK

TOP500 porovnává systémy podle výsledků benchmarku HPL/LINPACK (Rmax). **Green500** sleduje energetickou efektivitu. **Rpeak** je teoretický výkon, **Rmax** dosažená hodnota; oba nejsou automaticky měřítkem výkonu trénování neuronových sítí.

13.4 Případové studie z přednášek

Příklad	Význam / údaje ze snímků
Frontier (HPE Cray)	Exascale HPC; AMD EPYC + AMD Instinct akcelerátory; rozsáhlá síť a kapalinové chlazení. Uvedené výsledky odpovídají historickému stavu snímků.
Karolina (IT4Innovations)	Český HPC systém z roku 2021; univerzální i GPU části, v prezentaci 576× NVIDIA A100; rychlá síť a paralelní úložiště.
Meta AI Research SuperCluster	Příklad infrastruktury zaměřené na náročné trénování AI.
RCI ČVUT	Příklad národního výzkumného clusteru s GPU NVIDIA V100 a sítí InfiniBand.

Přednáška dále srovnává procesory **AMD EPYC, Intel Xeon Max, IBM Power10, Ampere Altra a NVIDIA Grace**. Není účelné memorovat jejich historické počty jader bez pochopení kompromisů: CPU výkon, paměťová propustnost, počet paměťových kanálů, PCIe, příkon, případné maticové akcelerátory.

> **VYSVĚTLENÍ NAVÍC – PROČ MÁ SÍŤ VÝZNAM**
>
> Při datovém paralelismu musí uzly sdílet aktualizace gradientů. Pokud výpočet jedné dávky trvá krátce, ale synchronizace dlouho, přidání dalších GPU nemusí zrychlovat lineárně. Proto jsou InfiniBand a jiné rychlé propoje zásadní součástí HPC systému.

KAPITOLA 14

Kvantové počítače – potenciál a limity

Základy qubitů a proč kvantový výpočet nelze chápat jako rychlejší klasickou GPU.

Ze zdrojů: 7APAI_03, sn. 25–36; 7APAI_01, sn. 18–20.

14.1 Bit, qubit a superpozice

Klasický bit nabývá hodnot 0 nebo 1. **Qubit** má obecně stav $|\psi = \alpha|0 + \beta|1$, kde α a β jsou komplexní amplitudy. Při měření v této bázi získáme 0 s pravděpodobností $|\alpha|^2$ a 1 s pravděpodobností $|\beta|^2$.

> **DŮLEŽITÉ UPŘESNĚNÍ K PREZENTACI**
>
> Správná normalizace qubitu je $\mathbf{|\alpha|^2 + |\beta|^2 = 1}$, nikoli obecně $\alpha + \beta = 1$. Superpozice neznamená, že můžeme jedním měřením přečíst všechny možnosti současně.

14.2 Provázání a interference

Kvantové provázání (entanglement) znamená, že stav více qubitů nelze obecně popsat jako součin jednotlivých nezávislých stavů. **Interference** umožňuje posilovat amplitudy některých výsledků a potlačovat jiné. Tyto jevy jsou klíčové pro kvantové algoritmy, ale zároveň jsou kvantové systémy citlivé na šum a chyby.

14.3 Vztah ke klasickému HPC a AI

Prezentace zmiňují hybridní kvantově-klasické výpočty: klasický HPC systém řídí a zpracovává běžné výpočty, kvantový procesor řeší určitou specializovanou část. Není prokázáno, že by současné kvantové počítače obecně rychleji či přesněji trénovaly běžné neuronové sítě než GPU. Potenciální výhoda vždy závisí na konkrétním algoritmu a podmínkách.

> **ZAPAMATUJ SI**
> Kvantový počítač není „GPU s qubity“. Rychlostní výhoda není univerzální a nelze ji odvodit samotným počtem qubitů.

KAPITOLA 15

Srovnání a výběr architektury

Přehled pro rychlé opakování před zkoušením i praktické návrhy.

Ze zdrojů: Syntéza všech devíti prezentací; rozhodovací postup je doplněn pro výuku.

15.1 Klíčová mapa architektur



Orientace v návrhovém prostoru: kompromis mezi flexibilitou, specializací, cenou a spotřebou.

Architektura	Změna funkce	Typická volba
CPU	Software	Řízení, univerzální aplikace, malé modely
GPU	Software/kernely	Velké tensorové výpočty; training + inference
TPU / NPU / VPU	Software v rámci podporovaných operací	Efektivní konkrétní ML inference nebo tensorové úlohy
FPGA	Překonfigurování hardwaru	Streaming, determinismus, prototyp ASIC
ASIC	Po výrobě zpravidla pevná logika	Velké série, maximalizace efektivity
SoC / SoM	Integrační úroveň, ne výpočetní ISA	Kompaktní embedded systémy
HPC cluster	Konfigurace a programy mnoha uzlů	Rozsáhlé modely, simulace, distribuovaný trénink

15.2 Jak vybrat řešení – pět otázek

- **Co počítám?** Trénování, jednorázová inference, video stream, řídicí smyčka?
- **Jak rychle to musí být?** Přípustná latence, FPS nebo denní objem.
- **Jaká omezení mám?** RAM, příkon, cena, chlazení, dostupná síť.
- **Co podporuje software?** Framework, export modelu, operátory, INT8/FP16, ovladače.
- **Jaká je životnost řešení?** Počet kusů, aktualizace, zabezpečení, servis.

15.3 Tři řešené situace

A. Kamera na stroji: musí detekovat poškozený kus offline s předvídatelnou latencí. Kandidáti: Coral Edge TPU (podporovaný INT8 model), Jetson (komplexnější vision pipeline), FPGA (specifická streamovací logika). Zvažujeme FPS, senzory, podporu modelu a energii.

B. Trénování rozsáhlé neuronové sítě: vysoký objem dat, týdny výpočtů. Kandidát: GPU/TPU cluster s vysokorychlostní sítí a úložištěm. Je třeba řešit paralelismus, checkpointy a náklady.

C. Monitorování teploty v hale: měření každou minutu, bateriové čidlo. Kandidát: jednoduchý MCU + Zigbee/LoRa/NB-IoT. Neuronová síť je volitelná jen tehdy, pokud pravidla nestačí.

KAPITOLA 16

Kontrolní otázky a odpovědi

Pro aktivní opakování: nejdříve otázky, potom stručný klíč.

Ze zdrojů: Otázky sestavené k celé sadě prezentací.

16.1 Otázky (zkus bez nápovědy)

01. Proč se AI často dobře paralelizuje?

02. Jak se liší trénování a inference?

03. Co je MAC a proč je důležité?

04. Proč nejsou 4 TOPS INT8 a 0,5 TFLOPS FP16 přímo srovnatelné?

05. Jaký je rozdíl latence a propustnosti?

06. Proč nepomůže nekonečně mnoho GPU vždy?

07. Co odlišuje ISA od mikroarchitektury?

08. Co je pipeline v CPU?

09. Pro co je vhodná GPU?

10. Jaký je rozdíl TPU a Edge TPU?

11. Co znamená NPU a VPU?

12. Co je ASIC a kdy se vyplácí?

13. Jak se liší LUT a flip-flop ve FPGA?

14. K čemu slouží PIP/SMIP ve FPGA?

15. SRAM vs antifuse konfigurace FPGA?

16. Jaké jsou hlavní kroky FPGA vývoje?

17. Co je SoC a co SoM?

18. Jak se liší MCU a MPU?

19. Proč používat Edge AI?

20. Co dělá kvantizace INT8?

21. Jaké hlavní jednotky má Jetson Nano?

22. Čím se liší Coral Dev Board a Dev Board Micro?

23. Jak se rozlišuje CIoT a IIoT?

24. Kdy volit LoRa místo Wi-Fi?

25. Co je kyberfyzikální systém?

26. Jaké jednotky má moderní mobilní SoC?

27. V čem je rozdíl Rpeak a Rmax?

28. Co omezuje distribuovaný trénink?

29. Jak se normalizuje stav qubitu?

30. Zaručuje kvantový počítač rychlejší běžné DL?

16.2 Stručný klíč

01. Proč se AI často dobře paralelizuje?

Mnoho stejných operací nad různými daty a tensorovými prvky lze počítat souběžně.

02. Jak se liší trénování a inference?

Trénování upravuje parametry; inference s naučenými parametry vytváří predikce.

03. Co je MAC a proč je důležité?

Multiply-accumulate: násobení s přičtením. Základ maticových operací a neuronových sítí.

04. Proč nejsou 4 TOPS INT8 a 0,5 TFLOPS FP16 přímo srovnatelné?

Jiný datový typ, jiná definice operace, metodika a podporované výpočty.

05. Jaký je rozdíl latence a propustnosti?

Latence: doba jedné úlohy; propustnost: počet úloh za čas.

06. Proč nepomůže nekonečně mnoho GPU vždy?

Sekvenční práce a režie komunikace omezují škálování (Amdahlův princip).

07. Co odlišuje ISA od mikroarchitektury?

ISA je softwarové instrukční rozhraní; mikroarchitektura je jeho fyzická implementace.

08. Co je pipeline v CPU?

Rozdělení zpracování instrukcí do etap, které se provádějí překrytě.

09. Pro co je vhodná GPU?

Pro vysoce paralelizovatelné tensorové a grafické výpočty, zejména DL.

10. Jaký je rozdíl TPU a Edge TPU?

Obecné cloudové TPU řeší výkonné tensorové úlohy; Edge TPU cílí na úspornou podporovanou inferenci.

11. Co znamená NPU a VPU?

NPU je akcelerátor neuronových sítí; VPU specializace na computer vision.

12. Co je ASIC a kdy se vyplácí?

Účelový čip; při stabilní funkci, velkých sériích nebo silných požadavcích na efektivitu.

13. Jak se liší LUT a flip-flop ve FPGA?

LUT realizuje kombinační logiku; flip-flop ukládá stav mezi hodinovými takty.

14. K čemu slouží PIP/SMIP ve FPGA?

Programovatelné body spojují logické bloky a segmenty propojovací sítě.

15. SRAM vs antifuse konfigurace FPGA?

SRAM je opakovaně konfigurovatelná, ale po ztrátě napájení se obnovuje; antifuse je jednorázová.

16. Jaké jsou hlavní kroky FPGA vývoje?

RTL/HDL, simulace, syntéza, place-and-route, časová analýza, bitstream a test.

17. Co je SoC a co SoM?

SoC je integrovaný čip; SoM modul/deska obsahující čip a podpůrné komponenty.

18. Jak se liší MCU a MPU?

MCU integruje řízení a periférie pro embedded/RT úlohy; MPU typicky slouží výkonnějším OS.

19. Proč používat Edge AI?

Nižší latence, provoz offline, menší přenos dat a lokální soukromí.

20. Co dělá kvantizace INT8?

Zmenšuje numerickou reprezentaci a výpočetní nároky; může ovlivnit přesnost modelu.

21. Jaké hlavní jednotky má Jetson Nano?

ARM Cortex-A57 CPU a GPU NVIDIA Maxwell se 128 CUDA jádry.

22. Čím se liší Coral Dev Board a Dev Board Micro?

Dev Board je robustnější SoM/MPU platforma; Micro mikrokontrolérová deska s FreeRTOS.

23. Jak se rozlišuje CIoT a IIoT?

Spotřebitelské vs průmyslové nasazení IoT.

24. Kdy volit LoRa místo Wi-Fi?

Pro malé zprávy a dlouhý dosah při nízké energii, nikoli velké video soubory.

25. Co je kyberfyzikální systém?

Propojení fyzického procesu se senzory, softwarem a řízením.

26. Jaké jednotky má moderní mobilní SoC?

CPU, GPU, NPU, ISP, DSP a komunikační modem podle konkrétního čipu.

27. V čem je rozdíl Rpeak a Rmax?

Rpeak je teoretický strop; Rmax naměřený výkon benchmarku LINPACK.

28. Co omezuje distribuovaný trénink?

Síťová komunikace, synchronizace, úložiště, paměť a sekvenční části.

29. Jak se normalizuje stav qubitu?

Součet druhých mocnin velikostí amplitud $|\alpha|^2 + |\beta|^2 = 1$.

30. Zaručuje kvantový počítač rychlejší běžné DL?

Ne; výhoda závisí na konkrétní úloze a známém kvantovém algoritmu.

KAPITOLA 17

Slovníček, poznámky a zdroje

Závěrečná část pro rychlé vyhledávání a ověření původních snímků.

Ze zdrojů: Mapování všech 9 přiložených přednášek.

17.1 Mini-slovníček

Termín	Význam
AI / ML / DL	umělá inteligence / strojové učení / hluboké učení
MAC	multiply-accumulate, násobení s akumulací
Tensor	vícerozměrné pole čísel
Inference	výpočet predikce z naučeného modelu
FLOPS / TOPS	operace v pohyblivé řádové čárce / biliony operací za sekundu
Throughput / latency	propustnost / odezva
CPU / GPU	centrální / grafický procesor
TPU / NPU / VPU	tensorová / neuronová / obrazová akcelerační jednotka
DSP / ISP	digitální signálový procesor / procesor obrazového signálu
ASIC / FPGA	účelový čip / programovatelná hradlová pole
LUT / RTL / HDL	vyhledávací tabulka / návrh na úrovni registrů / jazyk pro popis HW
SoC / SoM	systém na čipu / systém na modulu
MCU / MPU	mikrokontrolér / mikroprocesor
Edge AI / IoT	lokální AI na okraji sítě / internet věcí
HPC / HPL	vysoce výkonné výpočty / LINPACK benchmark
Rpeak / Rmax	teoretické maximum / naměřená výkonnost v LINPACKu
Qubit	kvantový bit, stav s amplitudami
CPS	kyberfyzikální systém

17.2 Přehled původních souborů

Soubor	Témata / relevantní snímky
7APAI_01.pdf	Základ AI HW; Edge AI; CPU/GPU/TPU/VPU; Jetson a Coral; ASIC — sn. 2–52.
7APAI_02_s.pdf	SoC, SoM, ARM/MIPS/x86, MCU, Apple, Raspberry Pi — sn. 2–39.
7APAI_03.pdf	HPC, CPU servery, Frontier, Karolina, kvantové výpočty — sn. 2–36.
7APAI_ASIC.pdf	Integrované obvody, typy ASIC, PLD/CPLD — sn. 2–20.
7APAI_FPGA.pdf	Konfigurace FPGA, LUT, routing, HDL a Verilog — sn. 2–29.
7APAI_Coral.pdf	Coral Dev Board Micro, Cortex-M, FreeRTOS, TFLite — sn. 2–15.
7APAI_Jetson_Nano.pdf	Platforma Jetson Nano, parametry, použití — sn. 2–11.
7APAI_IoT.pdf	IoT, sítě, IIoT a Průmysl 4.0 — sn. 3–31.
7APAI_Mob.pdf	Mobilní AI, Kirin, Exynos, Apple A17 — sn. 2–29.

17.3 Věcná upřesnění – co se neučit doslovně

- Srovnání výkonu bez datového typu (FP16, INT8) je zavádějící.
- Qubit: správná normalizace je $|\alpha|^2 + |\beta|^2 = 1$.
- Kvantový počítač není automaticky rychlejší pro jakoukoli AI úlohu.
- Jednotlivé FPGA/ASIC technologie nemají univerzální ekonomický bod zlomu.
- SoC, SoM, MCU, NPU a FPGA nejsou jedna vzájemně vylučující klasifikace.
- Produktové parametry, postavení v TOP500 i podpora software se v čase mění.

> **POZNÁMKA K AUTORSTVÍ A ROZSAHU**
>
> Základem byly výhradně přiložené prezentace vyučujícího **Mgr. Rostislava Fojtíka, Ph.D. (PřF OU)**. Učební vysvětlení, syntéza, řešené situace, orientační výpočty, otázky a upozornění jsou vlastní doplnění pro lepší porozumění; nenahrazují aktuální datasheet ani oficiální požadavky ke zkoušce.